

Real Python pandas Cheat Sheet: Detailed Edition

Practice at realpython.com with hands-on coding exercises, quizzes, and a personal AI mentor.

Getting Started

- Import pandas as `pd`, like everyone does
- A `DataFrame` is a table of `Series` columns that share one index
- pandas 3 needs Python 3.11 or newer and stores text as `str`
- All examples below use the sample `df` built in this section

Install pandas

```
$ python -m pip install pandas
$ python -m pip install pyarrow matplotlib
$ uv add pandas  # In a uv project
$ uv add pyarrow matplotlib
```

Create the Sample Data

```
import pandas as pd

df = pd.DataFrame({
    "date": pd.date_range(
        "2026-01-04", periods=6, freq="W",
    ),
    "region": [
        "North", "South", "North",
        "East", "South", "North",
    ],
    "product": [
        "Widget", "Gizmo", "Gizmo",
        "Widget", "Widget", "Gizmo",
    ],
    "units": [12, 7, 5, 9, 14, 3],
    "price": [2.5, 4.0, 4.0, 2.5, 2.5, 4.0],
})
df.to_csv("sales.csv", index=False)
```

Build a Series

```
s = pd.Series(
    [4, 19, 11],
    index=["Oslo", "Lima", "Rome"],
    name="temp",
)
s["Lima"]      # 19
s.index        # Index(['Oslo', 'Lima', ...])
s.to_numpy()   # array([ 4, 19, 11])
```

Build a DataFrame Other Ways

```
rows = [
    {"city": "Oslo", "temp": 4},
    {"city": "Lima", "temp": 19},
]
pd.DataFrame(rows)  # From records
pd.DataFrame(
    [[4, "Oslo"], [19, "Lima"]],
    columns=["temp", "city"],
)
```

Learn More on realpython.com/search:

`pandas dataframe` · `pandas`

Reading and Writing Data

- Every `pd.read_*()` reader has a matching `.to_*()` writer
- Pass `index=False` so the row index isn't written as a column
- Excel needs `openpyxl`, and Parquet needs `pyarrow`
- SQL takes a `sqlite3` connection or a SQLAlchemy engine

Read a CSV File With Options

```
sales = pd.read_csv(
    "sales.csv",
    usecols=["date", "region", "units"],
    parse_dates=["date"],
    dtype={"region": "category"},
    na_values=["n/a", "-"],
)
```

Read Big Files in Chunks

```
total = 0
for chunk in pd.read_csv(
    "sales.csv", chunksize=4,
):
    total += chunk["units"].sum()
total  # 50
```

Write and Read Excel

```
df.to_excel(
    "sales.xlsx", sheet_name="Q1",
    index=False,
)
pd.read_excel("sales.xlsx", sheet_name="Q1")
pd.read_excel("sales.xlsx", sheet_name=None)
# → {"Q1": DataFrame}, one per sheet
```

Write and Read JSON

```
df.to_json(
    "sales.json", orient="records",
    date_format="iso",
)
pd.read_json("sales.json")
df.to_json(
    "sales.jsonl", orient="records",
    lines=True, date_format="iso",
)
pd.read_json("sales.jsonl", lines=True)
```

Write and Read Parquet

```
df.to_parquet("sales.parquet")
pd.read_parquet(
    "sales.parquet",
    columns=["region", "units"],
)
```

Write and Read SQL

```
import sqlite3

con = sqlite3.connect("sales.db")
df.to_sql(
    "sales", con, index=False,
    if_exists="replace",
)
pd.read_sql(
    "SELECT * FROM sales WHERE units > ?",
    con, params=(8,),
)  # 3 rows
```

Compress on the Fly

```
df.to_csv("sales.csv.gz", index=False)
pd.read_csv("sales.csv.gz")  # Infers gzip
```

Learn More on realpython.com/search:

`pandas read_csv` · `pandas excel` · `pandas sql` · `pandas json`

Inspecting Data

- A non-null count below the row count means missing values
- `.describe()` covers numeric and date columns by default

Take a First Look

```
df.head(3)     # First 3 rows
df.tail()      # Last 5 rows
df.sample(2)   # 2 random rows
df.shape       # (6, 5)
df.columns     # Column labels
df.index       # RangeIndex(0, 6)
```

Check Types and Stats

```
df.info()      # Dtypes and non-null counts
df.dtypes      # region → str, units → int64
df.describe()  # count, mean, std, ...
df.memory_usage(deep=True)
```

Count Categories

```
>>> df["region"].value_counts()
region
North    3
South    2
East     1
Name: count, dtype: int64
```

Summarize a Column

```
df["units"].sum()       # 50
df["units"].mean()      # 8.333333333333334
df["units"].median()    # 8.0
df["region"].nunique()  # 3
df["region"].unique()   # North, South, East
```

Show More Rows and Columns

```
pd.set_option("display.max_columns", 50)
pd.set_option("display.precision", 2)
with pd.option_context(
    "display.max_rows", 10,
):
    print(df)
```

Learn More on realpython.com/search:

`explore dataset` · `pandas dtypes`

Selecting Data

- `.loc` uses labels, and its slices include the end
- `.iloc` uses positions, and its slices exclude the end
- `.at` and `.iat` get or set one cell quickly
- One label gives a `Series`, and a list gives a `DataFrame`

Pick Columns

```
df["units"]                  # Series
df[["region", "units"]]      # DataFrame
df.select_dtypes("number")   # units, price
df.filter(like="pr")         # product, price
```

Pick Rows and Cells by Label

```
df.loc[0]                # Row 0 as Series
df.loc[0, "region"]      # 'North'
df.loc[0:2, "units"]     # 12, 7, 5
df.loc[
    df["units"] > 10, ["region", "units"]
]  # Rows 0, 4
df.at[3, "units"]        # 9
```

Pick Rows and Cells by Position

```
df.iloc[0, 1]      # 'North'
df.iloc[:2, -2:]   # 2 rows, last 2 columns
df.iloc[-1]        # Last row
df.iat[3, 3]       # 9
```

Use a Meaningful Index

```
by_region = df.set_index("region")
by_region.loc["North", "units"]  # 12, 5, 3
by_region.reset_index()  # Back to a column
by_region.sort_index()   # East, North, ...
```

Learn More on realpython.com/search:

`pandas loc iloc` · `reset index`

Become a Python and AI expert with your free personal learning plan at realpython.com/learning-plan

Real Python

Working With a MultiIndex

- Pass a list to `.set_index()` to get one index level per column
- Sort the index before you slice it
- Grouping by several keys returns a `MultiIndex`, too

Build and Slice a MultiIndex

```
mi = df.set_index(["region", "product"])
mi = mi.sort_index()
mi.loc["North", "units"]  # 5, 3, 12
mi.loc[("North", "Gizmo"), "units"]  # 5, 3
mi.xs("Gizmo", level="product")  # 3 rows
```

Rearrange or Flatten It

```
mi.index.names  # ['region', 'product']
mi.swaplevel()  # product, then region
mi.reset_index()  # Both back to columns
mi.groupby(level="region")["units"].sum()
# → East 9, North 20, South 21
```

Learn More on realpython.com/search:

`multiindex` · `pandas index`

Filtering and Sorting

- Combine conditions with `&`, `|`, and `~`, and wrap each one in parentheses
- Use `@name` to reach Python variables in `.query()`

Filter With Boolean Masks

```
df[df["units"] > 10]            # 2 rows
df[
    (df["region"] == "North")
    & (df["units"] > 5)
]  # 1 row
df[~(df["region"] == "North")]  # 3 rows
df[df["units"].between(5, 10)]  # 3 rows
```

Filter by Membership or Text

```
df[df["product"].isin(["Widget"])]
df[df["product"].str.startswith("G")]
```

Filter With a Query String

```
min_units = 8
df.query("units >= @min_units")  # 3 rows
df.query("region == 'North' and units > 5")
df.query("product in ['Gizmo']")
```

Sort Rows

```
df.sort_values("units", ascending=False)
df.sort_values(
    ["region", "units"],
    ascending=[True, False],
)
df.nlargest(2, "units")    # Rows 4, 0
df.nsmallest(2, "units")   # Rows 5, 2
df["units"].rank(ascending=False)
```

Learn More on realpython.com/search:

`pandas filter` · `pandas sort`

Data Types and Conversion

- Integer columns with gaps load as `float64`
- Nullable `Int64`, `boolean`, and `string` keep `<NA>` instead
- Use `category` for a few labels that repeat often

Check and Convert Types

```
df.dtypes
df["units"].astype(float)
df.astype(
    {"units": "int32", "price": "float32"}
)
df["units"].astype(str)  # Text labels
```

Parse Messy Numbers and Dates

```
qty = pd.Series(["3", "n/a", "5"])
qty = pd.to_numeric(qty, errors="coerce")
# → 3.0, NaN, 5.0
pd.to_datetime(
    ["05/01/2026"], format="%d/%m/%Y",
)  # DatetimeIndex(['2026-01-05'])
```

Use Nullable Types

```
qty.astype("Int64")   # 3, <NA>, 5
df.convert_dtypes()   # Best nullable types
```

Use Ordered Categories

```
size = pd.Series(["M", "S", "L"]).astype(
    pd.CategoricalDtype(
        ["S", "M", "L"], ordered=True,
    )
)
size.sort_values()   # S, M, L
size.max()           # 'L'
```

Learn More on realpython.com/search:

`pandas dtypes` · `categorical data`

Cleaning Data

- Most methods return a new object, so assign the result back
- pandas 3 uses Copy-on-Write: chained assignment like `df["a"][0] = 1` never changes `df`, so use `.loc`

Messy Sample Data

```
raw = pd.DataFrame({
    "id": ["a1", "a2", "a2", "a3"],
    "qty": [3, None, None, 5],
    "city": [" oslo", "lima", "lima", None],
})
```

Find Missing Values

```
>>> raw.isna().sum()
id      0
qty     2
city    1
dtype: int64
```

Drop Missing Values

```
raw.dropna()                # 1 row left
raw.dropna(subset=["qty"])  # 2 rows left
raw.dropna(thresh=3)        # Needs 3 non-NA
raw.dropna(axis="columns")  # Only id
```

Fill Missing Values

```
raw["qty"].fillna(0)      # 3, 0, 0, 5
raw["qty"].fillna(raw["qty"].median())
# → 3, 4, 4, 5
raw["qty"].ffill()        # 3, 3, 3, 5
raw["qty"].interpolate()  # Linear fill
raw.fillna({"qty": 0, "city": "?"})
```

Remove Duplicates

```
raw.duplicated().sum()  # 1
raw.drop_duplicates()   # 3 rows
raw.drop_duplicates(subset="id", keep="last")
```

Rename, Drop, and Replace

```
df.rename(columns={"units": "qty"})
df.rename(columns=str.upper)
df.drop(columns=["price"])
df.drop(index=[0, 1])
df["region"].replace({"North": "N"})
df["units"].clip(upper=10)
```

Unify Inconsistent Labels

```
labels = pd.Series(["N", "North", "north "])
labels = labels.str.strip().str.title()
labels.replace({"N": "North"})
# → North, North, North
```

Set Values Safely

```
out = df.copy()
out.loc[0, "units"] = 13
out.loc[out["units"] > 10, "price"] = 2.0
```

Learn More on realpython.com/search:

`data cleaning` · `pandas dropna` · `missing data`

Working With Text

- `.str` runs string methods on every value and skips missing ones
- `.str.contains()` treats the pattern as a regex by default

Clean Up Strings

```
names = pd.Series([
    "  Ada Lovelace",
    "grace hopper ",
    "Guido van Rossum",
])
names = names.str.strip()
names.str.title()  # ..., 'Guido Van Rossum'
names.str.lower()
names.str.len()    # 12, 12, 16
```

Search and Match

```
names.str.contains("van")  # True for row 2
names.str.contains("ada", case=False)
names.str.startswith("G")  # True for row 2
names.str.count("a")       # 2, 1, 1
```

Split, Slice, and Extract

```
names.str.split().str[0]  # 'Ada', ...
names.str.split(" ", n=1, expand=True)
names.str[:3]             # 'Ada', 'gra', ...
names.str.extract(
    r"^(?P<first>\w+) (?P<last>.+)$"
)  # → first and last columns
```

Replace With a Regex

```
cash = pd.Series(["$1,200", "$950"])
(
    cash.str.replace(r"[$,]", "", regex=True)
    .astype(int)
)  # 1200, 950
```

One-Hot Encode Tags

```
tags = pd.Series(["a|b", "b"])
tags.str.get_dummies(sep="|")
# → Columns a, b with rows [1, 1], [0, 1]
```

Tidy Column Names

```
df.columns = (
    df.columns.str.strip()
    .str.lower()
    .str.replace(" ", "_")
)
```

Learn More on realpython.com/search:

`pandas string` · `regular expressions`

Become a Python and AI expert with your free personal learning plan at realpython.com/learning-plan

Real Python

Dates and Times

- Parse dates at load time with `parse_dates=`
- pandas 3 spells period ends `"ME"`, `"QE"`, and `"YE"`
- A `DatetimeIndex` allows slicing with partial date strings

Extract Date Parts

```
d = df["date"]
d.dt.year           # 2026, 2026, ...
d.dt.month          # 1, 1, 1, 1, 2, 2
d.dt.day_name()     # 'Sunday', ...
d.dt.quarter        # 1, 1, ...
d.dt.strftime("%b %d")  # 'Jan 04', ...
d.dt.to_period("M")     # 2026-01, ...
```

Do Date Arithmetic

```
(d - d.min()).dt.days  # 0, 7, ..., 35
d + pd.Timedelta(days=3)
d + pd.offsets.MonthEnd()  # 2026-01-31
```

Build Date Ranges

```
pd.date_range(
    "2026-01-01", periods=3, freq="D",
)
pd.date_range(
    "2026-01-01", periods=3, freq="ME",
)  # Month ends
pd.bdate_range("2026-01-01", "2026-01-07")
# → Weekdays only
```

Slice and Resample a Time Index

```
ts = df.set_index("date")
ts.loc["2026-01"]  # 4 January rows
ts.loc["2026-01-10":"2026-01-31"]
ts.resample("ME")["units"].sum()
# → 2026-01-31: 33, 2026-02-28: 17
```

Total by Calendar Period

```
df.resample("ME", on="date")["units"].sum()
# → 33, 17
d.dt.to_period("M").value_counts()
# → 2026-01: 4, 2026-02: 2
```

Combine Date Parts

```
parts = pd.DataFrame({
    "year": [2026, 2026],
    "month": [1, 2],
    "day": [5, 9],
})
pd.to_datetime(parts)  # Jan 5, Feb 9
```

Handle Time Zones

```
utc = d.dt.tz_localize("UTC")
utc.dt.tz_convert("Europe/Oslo")
```

Learn More on realpython.com/search:

`pandas datetime` · `time series`

Adding and Transforming Columns

- Vectorized math on whole columns is fast
- `np.where()` and `np.select()` handle conditions
- Reach for `.apply()` only when nothing vectorized fits

Compute a New Column

```
df["revenue"] = df["units"] * df["price"]
# → 30.0, 28.0, 20.0, 22.5, 35.0, 12.0
```

Apply Conditions

```
import numpy as np

np.where(df["units"] > 8, "high", "low")
np.select(
    [df["units"] > 10, df["units"] > 5],
    ["high", "mid"], default="low",
)  # high, mid, low, mid, high, low
df["units"].where(df["units"] > 5, 0)
# → 12, 7, 0, 9, 14, 0
```

Map and Bin Values

```
codes = {
    "North": "N", "South": "S", "East": "E",
}
df["region"].map(codes)  # N, S, N, ...
pd.cut(
    df["units"], bins=[0, 5, 10, 15],
    labels=["S", "M", "L"],
)
pd.qcut(
    df["units"], q=2, labels=["low", "high"],
)
```

Apply a Function

```
df["units"].apply(lambda x: x ** 2)
df.apply(
    lambda row: row["units"] * row["price"],
    axis=1,
)  # Slow, but same result as revenue
```

Learn More on realpython.com/search:

`pandas apply` · `numpy`

Grouping and Aggregating

- `.groupby()` splits rows, applies a function, and combines the results
- `.transform()` returns one value per original row
- Pass `observed=True` when grouping by a categorical

Aggregate One Column

```
>>> df.groupby("region")["units"].sum()
region
East      9
North    20
South    21
Name: units, dtype: int64
```

Compute Several Stats

```
df.groupby("region")["units"].agg(
    ["sum", "mean", "max"]
)
df.groupby("region").agg(
    total=("revenue", "sum"),
    orders=("units", "count"),
)  # Named aggregation
```

Group by Several Keys

```
df.groupby(
    ["region", "product"], as_index=False,
)["units"].sum()
# → 5 rows with flat columns
df.groupby("region").size()  # 1, 3, 2
```

Transform and Filter Groups

```
g = df.groupby("region")["units"]
g.transform("sum")  # 20, 21, 20, 9, 21, 20
df["units"] / g.transform("sum")  # Share
g.cumcount()        # 0, 0, 1, 0, 1, 2
df.groupby("region").filter(
    lambda grp: len(grp) > 1
)  # Drops East
```

Find the Top Row per Group

```
idx = g.idxmax()  # East 3, North 0, ...
df.loc[idx]       # Rows 3, 0, 4
```

Group by Time

```
months = df["date"].dt.month
df.groupby(months)["units"].sum()  # 33, 17
```

Loop Over Groups

```
for region, grp in df.groupby("region"):
    print(region, len(grp))
# East 1, North 3, South 2
```

Learn More on realpython.com/search:

`pandas groupby` · `aggregate data`

Reshaping Data

- `.pivot()` needs unique index and column pairs, while `.pivot_table()` aggregates duplicates
- `.melt()` turns wide data into long data
- `.stack()` moves columns into the index, and `.unstack()` reverses it

Build a Pivot Table

```
df.pivot_table(
    index="region", columns="product",
    values="units", aggfunc="sum",
    fill_value=0, margins=True,
)  # Adds an "All" row and column
```

Cross-Tabulate Counts

```
>>> pd.crosstab(df["region"], df["product"])
product  Gizmo  Widget
region
East         0       1
North        2       1
South        1       1
```

Melt Wide Data Into Long Data

```
wide = pd.DataFrame({
    "city": ["Oslo", "Lima"],
    "jan": [-4, 23], "feb": [-3, 24],
})
long = wide.melt(
    id_vars="city", var_name="month",
    value_name="temp",
)  # 4 rows
```

Pivot Long Data Back to Wide

```
long.pivot(
    index="city", columns="month",
    values="temp",
)  # Columns: feb, jan
```

Stack and Unstack

```
st = wide.set_index("city").stack()
# → (Oslo, jan) -4, (Oslo, feb) -3, ...
st.unstack()  # Back to wide
by = ["region", "product"]
(
    df.groupby(by)["units"].sum()
    .unstack(fill_value=0)
)
```

Explode List Values Into Rows

```
tags = pd.DataFrame({
    "k": ["a", "b"],
    "tags": [["x", "y"], ["z"]],
})
tags.explode("tags")  # 3 rows
```

Learn More on realpython.com/search:

`pivot table` · `reshape data`

Become a Python and AI expert with your free personal learning plan at realpython.com/learning-plan

Real Python

Combining DataFrames

- `merge()` joins on columns, `.join()` on the index, and `concat()` stacks
- `validate=` catches unexpected duplicate keys

Lookup Table for the Examples

```
managers = pd.DataFrame({
    "region": ["North", "South", "West"],
    "manager": ["Ana", "Bo", "Cy"],
})
```

Merge on a Key Column

```
df.merge(managers, on="region")  # 5 rows
df.merge(managers, on="region", how="left")
# → 6 rows, NaN for East
df.merge(managers, on="region", how="outer")
# → 7 rows
```

Track and Validate Matches

```
m = df.merge(
    managers, how="outer", indicator=True,
)
m["_merge"].value_counts()
# → both 5, left_only 1, right_only 1
df.merge(
    managers, how="left",
    validate="many_to_one",
)
```

Find Rows Without a Match

```
df.merge(managers, how="left_anti")
# → East row only, new in pandas 3.0
```

Merge Keys With Different Names

```
staff = managers.rename(
    columns={"region": "area"}
)
df.merge(
    staff, left_on="region", right_on="area",
)
```

Join on the Index

```
df.join(
    managers.set_index("region"),
    on="region",
)  # Left join by default
```

Merge on the Nearest Key

```
rates = pd.DataFrame({
    "date": pd.to_datetime(
        ["2026-01-01", "2026-01-20"]
    ),
    "rate": [1.1, 1.2],
})
pd.merge_asof(df, rates, on="date")
# → rate: 1.1, 1.1, 1.1, 1.2, 1.2, 1.2
```

Concatenate Rows or Columns

```
pd.concat([df, df], ignore_index=True)
# → 12 rows, index 0 to 11
pd.concat(
    [df.head(2), df.tail(2)],
    keys=["first", "last"],
)
pd.concat(
    [df[["units"]], df[["price"]]],
    axis="columns",
)
```

Learn More on realpython.com/search:

`pandas merge` · `pandas concat`

Window Functions

- `.rolling(n)` returns `NaN` until it has `n` values, unless you set `min_periods`
- Time windows like `"14D"` need a `DatetimeIndex`

Shift and Compare Rows

```
u = df["units"]
u.shift(1)      # NaN, 12, 7, 5, 9, 14
u.diff()        # NaN, -5, -2, 4, 5, -11
u.pct_change()  # NaN, -0.42, -0.29, ...
```

Keep Running Totals

```
u.cumsum()    # 12, 19, 24, 33, 47, 50
u.cummax()    # 12, 12, 12, 12, 14, 14
u.expanding().mean()  # 12, 9.5, 8, ...
```

Roll a Window Over Rows

```
u.rolling(3).mean()  # NaN, NaN, 8, 7, ...
u.rolling(3, min_periods=1).sum()
# → 12, 19, 24, 21, 28, 26
u.rolling(3).agg(["min", "max"])
u.ewm(span=3).mean()  # Exponential weights
```

Use Time-Based Windows

```
ts = df.set_index("date")["units"]
ts.rolling("14D").sum()
# → 12, 19, 12, 14, 23, 17
```

Compute Windows per Group

```
g = df.groupby("region")["units"]
g.shift()   # Previous value in region
g.cumsum()  # 12, 7, 17, 9, 21, 20
```

Learn More on realpython.com/search:

`pandas rolling` · `time series`

Method Chaining

- Wrap a chain in parentheses and put one step per line
- A `lambda` in `.assign()` sees the DataFrame at that step
- pandas 3 adds `pd.col()` to replace many of those lambdas

Chain Steps Into a Pipeline

```
(
    df.query("units > 4")
    .assign(
        revenue=lambda d: (
            d["units"] * d["price"]
        )
    )
    .groupby("region")["revenue"].sum()
    .sort_values(ascending=False)
)  # South 63.0, North 50.0, East 22.5
```

Use Column Expressions

```
df.assign(
    revenue=pd.col("units") * pd.col("price")
)
df.loc[pd.col("units") > 10]  # Rows 0, 4
```

Pipe Into Your Own Functions

```
def add_revenue(d, col="revenue"):
    return d.assign(
        **{col: d["units"] * d["price"]}
    )

df.pipe(add_revenue, col="rev")
```

Select With Callables

```
df.loc[lambda d: d["units"] > 10]
(
    df.groupby("region")["units"].sum()
    .reset_index(name="total")
)  # Columns: region, total
```

Learn More on realpython.com/search:

`method chaining` · `pandas tricks`

Plotting

- `.plot()` draws with Matplotlib and returns its `Axes`
- Notebooks show plots inline, but scripts need `plt.show()`

Pick a Plot Kind

```
df.plot(x="date", y="units")  # Line
df["units"].plot(kind="hist", bins=5)
df.plot.scatter(x="price", y="units")
df["product"].value_counts().plot.pie()
df.plot.box(column="units", by="region")
```

Plot Grouped Results

```
(
    df.groupby("region")["units"].sum()
    .plot.bar(rot=0, title="Units")
)
```

Customize and Save

```
import matplotlib.pyplot as plt

ax = df.plot(
    x="date", y="units",
    figsize=(6, 3), title="Units",
)
ax.set_ylabel("Units sold")
plt.tight_layout()
plt.savefig("units.png", dpi=150)
```

Learn More on realpython.com/search:

`pandas plot` · `matplotlib`

Sharing Results

- `.to_markdown()` needs `tabulate`, and `.style` needs `jinja2`
- Styled tables render in notebooks and export to HTML or Excel

Export Tables as Text

```
print(df.to_string(index=False))
df.to_markdown(index=False)
df.to_html("sales.html", index=False)
df.to_dict("records")  # List of dicts
```

Style a Table

```
(
    df.style
    .format({"price": "${:.2f}"})
    .highlight_max(subset=["units"])
)
```

Write Several Excel Sheets

```
totals = df.groupby("region")["units"].sum()
with pd.ExcelWriter("report.xlsx") as xw:
    df.to_excel(
        xw, sheet_name="raw", index=False,
    )
    totals.to_excel(xw, sheet_name="summary")
```

Learn More on realpython.com/search:

`pandas excel` · `jupyter notebook`

Become a Python and AI expert with your free personal learning plan at realpython.com/learning-plan

Real Python

Performance Tips

- Vectorized column math beats Python loops by orders of magnitude
- `.iterrows()` builds a `Series` per row, so it's slow and loses dtypes
- Load less: pick columns, set dtypes, read in chunks, or use Parquet

Vectorize Instead of Looping

```
# Slow: a Python-level loop
total = [
    r["units"] * r["price"]
    for _, r in df.iterrows()
]
# Fast: one vectorized expression
total = df["units"] * df["price"]
```

Loop With .itertuples() if You Must

```
for row in df.itertuples(index=False):
    print(row.region, row.units)
```

Use Categories for Repeated Labels

```
cat = df["region"].astype("category")
cat.cat.categories  # East, North, South
cat.cat.codes       # 1, 2, 1, 0, 2, 1
cat.memory_usage(deep=True)  # Smaller
```

Downcast Numbers

```
pd.to_numeric(
    df["units"], downcast="integer",
)  # int8
```

Try the PyArrow Backend

```
pd.read_csv(
    "sales.csv", engine="pyarrow",
    dtype_backend="pyarrow",
)  # int64[pyarrow], string[pyarrow], ...
```

Load Only What You Need

```
pd.read_csv(
    "sales.csv",
    usecols=["region", "units"],
    dtype={"region": "category"},
)
pd.read_parquet(
    "sales.parquet", columns=["units"],
)
```

Learn More on realpython.com/search:

`pandas performance` · `pandas iterate` · `polars`

Common Gotchas

- Use `&`, `|`, and `~` because `and`, `or`, and `not` don't work element-wise
- `in` checks a Series' index, not its values
- Missing values never equal anything, including themselves

Combine Masks With & and |

```
df[(df["units"] > 5) & (df["price"] > 3)]
# df[df["units"] > 5 and ...]  → ValueError
```

Test Membership in Values

```
12 in df["units"]              # False
12 in df["units"].values       # True
df["units"].isin([12]).any()   # True
```

Detect Missing Values

```
np.nan == np.nan  # False
s = pd.Series([1, None])
s.dtype           # float64, not int64
s.isna()          # False, True
```

Avoid Chained Assignment

```
out = df.copy()
out["units"][0] = 99      # Warns, no effect
out.loc[0, "units"] = 99  # Works
```

Compare DataFrames

```
df.equals(df.copy())  # True
other = df.copy()
other.loc[1, "units"] = 8
df.compare(other)     # Shows 7 vs 8
pd.testing.assert_frame_equal(df, df)
```

Learn More on realpython.com/search:

`settingwithcopywarning` · `pandas tricks`

Quick Reference

Aggregation Names

Name	Returns
`"sum"`, `"prod"`	Total, product
`"mean"`, `"median"`	Average, middle value
`"min"`, `"max"`	Smallest, largest
`"count"`, `"size"`	Non-null count, row count
`"nunique"`	Distinct values
`"std"`, `"var"`	Spread
`"first"`, `"last"`	First, last value per group
`"idxmax"`, `"idxmin"`	Label of max, min

Frequency Aliases

Alias	Meaning
`"s"`, `"min"`, `"h"`	Second, minute, hour
`"D"`, `"B"`	Calendar day, business day
`"W"`, `"W-MON"`	Week ending Sunday, Monday
`"ME"`, `"MS"`	Month end, month start
`"QE"`, `"YE"`	Quarter end, year end

Common Dtypes

Dtype	Holds
`int64`, `Int64`	Integers, nullable integers
`float64`	Decimals and `NaN`
`bool`, `boolean`	Booleans, nullable Booleans
`str`	Text, the pandas 3 default
`category`	A few repeated labels
`datetime64[us]`	Dates and times

Merge Types

how=	Keeps
`"inner"`	Keys found in both, the default
`"left"`, `"right"`	All keys from one side
`"outer"`	All keys from both sides
`"cross"`	Every pair of rows
`"left_anti"`	Left rows with no match, new in 3.0

File Formats

Read, write	Needs
`read_csv()`, `.to_csv()`	Nothing extra
`read_json()`, `.to_json()`	Nothing extra
`read_excel()`, `.to_excel()`	`openpyxl`
`read_parquet()`, `.to_parquet()`	`pyarrow`
`read_sql()`, `.to_sql()`	`sqlite3` or SQLAlchemy

Do you want to go deeper on pandas?

Level up your skills with curated resources at Real Python:

- Coding exercises
- Interactive quizzes
- Learning paths
- Video courses
- Written tutorials
- Reference articles
- Podcast interviews

Become a Python and AI expert with your free personal learning plan at **realpython.com/learning-plan**